Data Mining and Knowledge Discovery (2025) 39:65
https://doi.org/10.1007/s10618-025-01147-x

Gradient boosted bagging for evolving data stream regression

Nuwan Gunasekara[1,2] · Bernhard Pfahringer[2] · Heitor Murilo Gomes[3] · Albert Bifet[2,4]

Received: 8 June 2025 / Accepted: 20 July 2025 / Published online: 6 August 2025
© The Author(s) 2025

Abstract

Gradient boosting has been extensively studied in batch learning. Recently, its streaming adaptation, Streaming Gradient Boosted Trees (SGBT), has surpassed existing state-of-the-art random subspace and random patches methods for streaming classification under various drift scenarios. However, its application in streaming regression remains unexplored. Vanilla SGBT with squared loss exhibits high variance when applied to streaming regression problems. To address this, we utilize bagging streaming regressors in this work to create Streaming Gradient Boosted Regression (SGBR). Bagging streaming regressors are employed in two ways: first, as base learners within the existing SGBT framework, and second, as an ensemble method that aggregates multiple SGBTs. Our extensive experiments on 11 streaming regression datasets, encompassing multiple drift scenarios, demonstrate that the SGB(OZA), a variant of the first SGBR category, significantly outperforms current state-of-the-art streaming regression methods in terms of both predictive power and computational cost.

Keywords Stream learning · Gradient boosting · Gradient boosted bagging · Gradient boosted regression · Data stream boosting · Streaming regression · Streaming gradient boosting · Streaming gradient boosted bagging · Streaming gradient boosted regression

Bernhard Pfahringer, Heitor Murilo Gomes, and Albert Bifet have contributed equally to this work.

Extended author information available on the last page of the article

1 Introduction

In the era of big data, the ability to perform regression on streaming data has become increasingly important. Data stream regression is crucial for applications where data continuously flows and predictions must be made in real-time, such as financial markets (Kovacs et al. 2021), sensor networks (Halstead et al. 2022), and online energy prediction (Oprea et al. 2019; Lobo et al. 2020; Wu et al. 2023). Unlike traditional batch learning, data stream regression must handle evolving data distributions (concept drifts) using limited memory and processing power (Bifet et al. 2018; Gomes et al. 2017, 2020). This makes it a challenging yet essential task for modern machine learning systems.

Ensemble methods have emerged as the dominant approach in data stream regression, consistently outperforming single-learner regressors. Techniques like bagging, random subspaces, efficient drift detection, and adaptation have improved predictive performance and adaptability to concept drifts (Gomes et al. 2018, 2020). The Adaptive Random Forest Regressor (ARFREG) (Gomes et al. 2018) and Self-Optimising K-Nearest Leaves (SOKNL) (Sun et al. 2022) are notable examples that leverage these ensemble strategies to achieve higher accuracy and robustness. These methods train multiple base learners on different subsets of data, incorporate advanced mechanisms such as drift detection, and prepare learners in the background to enhance their performance in a streaming environment.

While ensemble methods have shown great success in data stream regression, the potential of boosting-based methods has been relatively unexplored. Historically, boosting methods may have underperformed in streaming classification, leading to a lack of focus on their potential for regression tasks. However, recent advancements in gradient-boosted techniques, particularly the Streaming Gradient Boosted Trees (SGBT) (Gunasekara et al. 2024) method, have shown that boosting can surpass bagging and random forest-based methods in streaming classification. This opens up a world of possibilities, suggesting a promising potential for gradient-boosted approaches for data stream regression. This work presents:

- A novel gradient-boosted approach for datastream regression, termed as Streaming Gradient Boosted Regression (SGBR). Inspired by early batch boosting for regression, where bagging base learners were utilized in a gradient-boosted setup to reduce high variance (Friedman et al. 2000), the proposed SGBR leverages streaming bagging regressors in two distinct ways: firstly, as base learners within the existing SGBT framework, and secondly, as an ensemble method that aggregates multiple SGBTs. The first approach is referred to as Streaming Gradient Boosted Bags (SGB(Bag)), and the second approach is referred to as Bagged Streaming Gradient Boosted Trees (Bag(SGBT)).
- The vanilla SGB(Bag) employs streaming bagging regressors to yield SGB(OZA). This approach is extended to incorporate streaming random subspace and random patches techniques, yielding SGB(ARF) and SGB(SRP), respectively. Conversely, the vanilla Bag(SGBT) variant, OZA(SGBT), utilizes streaming bagging method to aggregate multiple SGBTs. Its extended version, SRP(SGBT), aggregates multiple SGBTs using a random patches approach.

- Experimental results on 11 streaming regression benchmarks, encompassing various concept drift scenarios, demonstrate that SGBR enhances the predictive capability of streaming bagging, random subspace, and random patches regressors. This performance improvement is achieved without significant computational overhead for SGB(Bag) configurations using bagging and random subspace techniques. Furthermore, SGB(OZA) surpasses the current state-of-the-art streaming regression method, SOKNL (Sun et al. 2022), by a significant margin in terms of both predictive power and computational efficiency.

While our approach draws conceptual inspiration from a technique proposed 25 years ago for batch gradient boosting regression (Friedman et al. 2000), it is important to note that this idea has not been adopted or explored in the context of streaming regression despite the emergence of online boosting algorithms over two decades ago. To the best of our knowledge, this is the first effort to bridge the gap between bagging and boosting in streaming regression by revisiting and adapting a historically underutilized technique for modern, evolving data streams. Our results show that this reinterpretation yields substantial performance gains over state-of-the-art streaming regression models-both in terms of predictive power and computational efficiency.

The paper is structured as follows. The next section reviews the current state-of-the-art ensemble methods and techniques for data stream learning. The following section introduces the proposed SGBR, which leverages streaming gradient boosting and streaming bagging for data stream regression. The experiments section details the experimental setup, where various SGBR variants are compared against leading streaming regressors. The conclusions are presented in the final section.

2 Related work

Bagging and boosting are two popular ensemble learning techniques in machine learning. Bagging randomly samples instances with replacement to train each member of the ensemble, while boosting attempts to boost the performance of the next base learner by considering the loss of the previous one, combining weak learners additively to produce a strong learner (Friedman et al. 2000; Friedman 2001). AdaBoost (Freund and Schapire 1997) is a notable boosting algorithm that weights misclassified instances more heavily to improve subsequent base learners. Gradient boosting extends this approach by using gradient information of the loss function to guide the training of subsequent learners (Friedman et al. 2000). XGBoost (Chen and Guestrin 2016) implements an efficient version of gradient boosting with specialized features like cache-aware processing and parallelization for batch learning.

In contrast to batch learning, Stream Learning models learn from evolving data streams (non-IID data), processing instances sequentially while maintaining limited memory and processing resources (Bifet et al. 2018; Gomes et al. 2017). A key challenge in stream learning is adapting to concept drifts - changes in the underlying data distribution (Bifet et al. 2018; Bifet and Gavalda 2007).

Data stream adaptation of bagging presents unique challenges due to the sequential nature of data. Oza Bag (OzaBag) (Oza and Russell 2001) was inspired by the observation that a binomial distribution can be approximated by a Poisson distribution as the number of instances grows large. Since batch bagging selects examples with probability $1/n$, OzaBag approximates this with *Poisson*(1) sampling.

Streaming ensemble methods can also leverage feature subspaces (random subspaces) or both instance and feature subsets (random patches) (Gomes et al. 2020). Notable streaming implementations include Adaptive Random Forest (Arf) (Gomes et al. 2017) and Streaming Random Patches (Srp) (Gomes et al. 2019), which use efficient base learners like Hoeffding Tree (HT) (Domingos and Hulten 2000) and Extremely Fast Decision Tree (EFDT) (Manapragada et al. 2018) in conjunction with drift detectors like ADWIN (Bifet and Gavalda 2007).

Boosting in data streams is particularly challenging due to the evolving nature of the data stream. Oza Boost (OzaBoost) (Oza and Russell 2001) tracks weights of correctly and incorrectly classified examples to compute the Poisson parameter λ. Online Smooth Boost (OSB) (Chen et al. 2012) extends this by using smooth distributions that avoid excessive weighting of individual examples.

Recent gradient boosting adaptations for streams include Axgb (Montiel et al. 2020) and AdIter (Wang et al. 2022). The state-of-the-art Sgbt (Gunasekara et al. 2024) has shown superior performance over previous boosting methods for data stream classification.

Many data stream regression methods exist in the literature (Choudhary et al. 2021). Single tree methods include Hoeffding Tree Regressor (HTR) (an adaptation of HT for regression) and Fast Incremental Regression Trees with Drift Detection (FIRT-DD) (Fast Incremental Model Trees with Drift Detection (FIMT-DD) is its classification counterpart) (Ikonomovska et al. 2011).

Streaming ensemble methods have shown superior performance for streaming regression (Gomes et al. 2018; Sun et al. 2022). ArfReg (Gomes et al. 2018) uses an ensemble of FIRT-DDs trained on feature subsets, while Online Extra Trees (OXT) (Mastelini et al. 2022) is a more recent tree-based ensemble. However, it did not show a statistically significant improvement over a resource-efficient ArfReg version (Mastelini et al. 2022). SOKNL (Sun et al. 2022) combines k-NN with ArfReg, using only the k most relevant trees for prediction, and has shown superior accuracy over ArfReg (Sun et al. 2022).

Despite significant advancements in bagging and random forest-based methods for data stream regression (Sun et al. 2022), boosting-based methods have received less attention. This may be due to their historical underperformance in classification settings until recently (Gunasekara et al. 2024). However, the success of Sgbt in surpassing bagging methods for stream classification (Gunasekara et al. 2024) suggests a promising potential for gradient boosting in data stream regression. Figure 1 summarizes the stream learners discussed in this work.

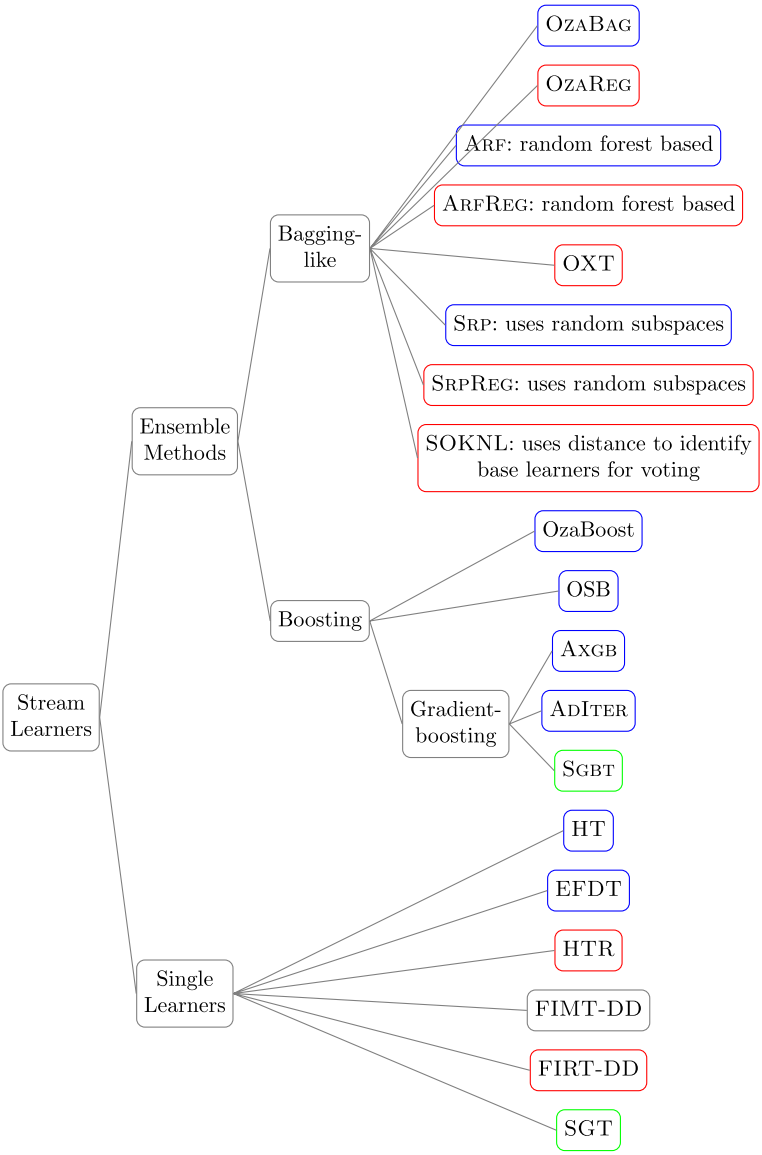


Fig. 1 Different existing stream learners discussed in this work. Colour scheme: classifiers, regressors , classifiers and regressors

3 Streaming gradient boosted regression

For a dataset with n instances, let x_i be the features for the i-th instance and y_i be its relevant target value. In gradient boosting, a model ϕ can be represented as S additive functions:

$$\hat{y}_i = \phi(x_i) = \sum_{s=1}^{S} f_s(x_i), f_s \in \mathcal{F}$$

to predict $\hat{y}_i$ (Friedman 2002; Chen and Guestrin 2016). Here, $\mathcal{F}$ is the space of regressors. The learning objective is to minimize:

$$\mathcal{L}(\phi) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{s=1}^{S} \Omega(f_s) \tag{1}$$

Where Ω penalizes the complexity of the *f* regressor. Here, *l* is a differentiable convex loss function which measures the difference between the prediction $\hat{y}_i$ and the target y_i. Furthermore, the loss at the s-th step is the loss incurred by the previous $(s-1)$ step and the loss incurred by tree f_s plus the regularization term:

$$\mathcal{L}^{(s)} = \sum_{i=1}^{n} l(y_i, \hat{y}^{(s-1)} + f_s(x_i)) + \Omega(f_s)$$

This loss can be approximated using second-order Taylor approximation (Chen and Guestrin 2016) to:

$$\mathcal{L}^{(s)} \simeq \sum_{i=1}^{n} \left[l(y_i, \hat{y}^{(s-1)}) + g_i f_s(x_i) + \frac{1}{2} h_i f_s^2(x_i) \right] + \Omega(f_s)$$

Here, $g_i = \partial_{\hat{y}^{(s-1)}} l(y_i, \hat{y}^{(s-1)})$ and $h_i = \partial^2_{\hat{y}^{(s-1)}} l(y_i, \hat{y}^{(s-1)})$ are the first and second order (hessian) gradient statistics of the loss considering $s-1$-th prediction. It can be rewritten (Chen and Guestrin 2016) to explain it as a weighted squared loss with weight h_i and target g_i/h_i:

$$\sum_{i=1}^{n} \frac{1}{2} h_i (f_t(x_i) - g_i/h_i)^2 + \Omega(f_s) + constant. \tag{2}$$

This weighted squared loss with *hessian* as the *weight* and *gradient* over *hessian* as the *target*, considering the previous boosting step's loss, was first introduced in Friedman et al. (2000). SGBT utilizes above weighted squared loss in equation 2 for streaming classification (Gunasekara et al. 2024). It provides SGBT the flexibility to utilize various streaming regression methods instead of the one employed in XGBOOST.

SGBT also utilizes streaming regressors with internal Base Learner Replacement strategy to adapt to concept drifts. Furthermore, each base learner in SGBT is trained on a subset of features (random subspace) to induce diversity into the ensemble.

Considering its success in streaming classification, one would naturally be inclined to use vanilla SGBT with squared loss for *l* on streaming regression. However, early experiments in Table 1 suggests that SGBT with 100 boosting iterations (100 FIRT-

Table 1 SGBT with 100 boosting iterations (100 FIRT-DDs as base learners) against single tree regressors (Adjusted Coefficient of Determination ($\bar{R}^2$))

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	**avg**	**avgRank**
FIRT-DD	0.60	**0.46**	**0.16**	0.75	0.24	−0.00	**0.46**	0.63	**0.57**	**0.58**	**0.42**	**0.4**	**1.4**
HTR	**0.65**	0.36	0.14	0.56	**0.25**	**0.04**	0.44	**0.64**	0.47	0.42	0.40	0.4	1.8
SGT(Gouk et al. 2019)	0.39	0.35	−2.50	0.62	−0.72	−0.04	−0.48	0.55	0.34	0.31	0.28	−0.1	3.5
SGBT	0.46	0.18	0.08	**0.80**	0.19	−0.05	0.38	0.49	0.42	0.42	0.22	0.3	3.2

The winning method for each dataset is bolded

(Please refer to Sect. 4.1 for parameter configurations of FIRT-DD, HTR, and SGT, with default SGT parameters from Gouk et al. (2019).)

(Experimental settings are detailed in Sect. 1)

DDs as base learners) performs worse than most of the single tree regressors. As a next step, one would hope to increase the predictive power of SGBT on streaming regression problems by adding more boosting iterations.

However, previous work on batch setting suggests that adding more boosting iterations increase variance on regression problems (Friedman et al. 2000). Our early experiments also confirm this with SGBT on streaming regression. Figure 2 clearly shows the increase in variance with more boosting iterations without much effect on bias for SGBT on Ailerons dataset. This compelled one to better understand bias and variance on ensemble methods.

3.1 Bias and variance

The literature explains two main concepts: *optimal prediction* and *main prediction* prior introducing bias and variance (Domingos 2000). The *optimal prediction* (y_*) (Domingos 2000) for an example x is the prediction that minimizes $\mathbb{E}_y[L(y, y_*)]$, where the subscript y denotes that the expectation is taken with respect to all possible *true values* y, weighted by their probabilities given x. The *main prediction* ($\hat{y}_m$):

$$\hat{y}_m = y_m^{\mathcal{L},\mathcal{D}} = \arg\min_{\acute{y}} \mathbb{E}_{\mathcal{D}}[L(\hat{y}, \acute{y})]$$

is the prediction value $\acute{y}$ with the minimum average loss across all predictions on $\mathcal{D}$ set of training sets (Domingos 2000), i.e., the prediction that **differs least** from all the predictions. The *main prediction* under squared loss is the mean of predictions; under absolute loss it is the median and under zero–one loss it is the mode (the most frequent prediction) (Domingos 2000).

The bias of a learner on an example x is:

$$\text{Bias}(x) = L(y_*, \hat{y}_m)$$

Domingos (2000). In words, the bias is the loss incurred by the *main prediction* relative to the *optimal prediction*.

The variance of a learner on an example x is

$$\text{Variance}(x) = \mathbb{E}_D[L(\hat{y}_m, y)].$$

Domingos (2000). The variance is the average loss incurred by predictions relative to the *main prediction* (Domingos 2000). Bias and variance may be averaged over all examples.

The *main prediction* ($\hat{y}_m$) measures the "central tendency" of a learner (Domingos 2000). Thus $\text{Bias}(x)$ measures the systematic loss incurred by a learner, and $\text{Variance}(x)$ measures the loss incurred by its fluctuations around the central tendency in response to different training sets. The bias is independent of the training set, and is zero for a learner that always makes the optimal prediction (Domingos 2000). The variance is independent of the true value of the predicted variable, and is

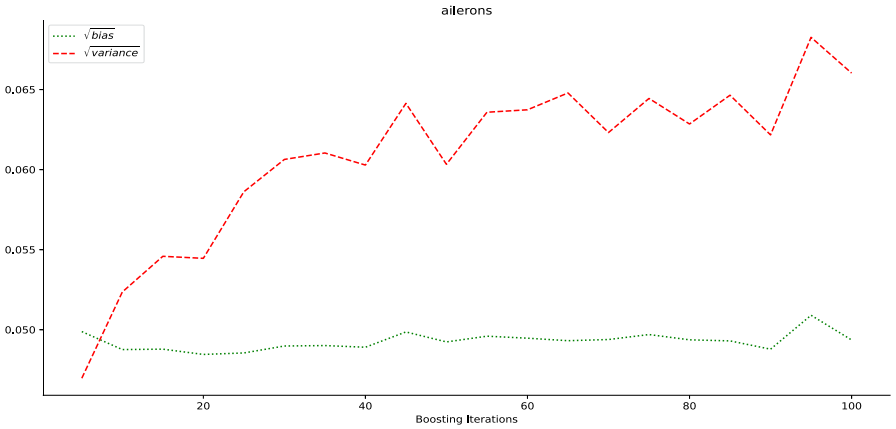


Fig. 2 Bias and variance of SGBT for different boosting iterations on Ailerons dataset. Green: $\sqrt{bias}$, red: $\sqrt{variance}$. (Please refer to sec 4.2 and Sect 1 for experimental configuration)

zero for a learner that always makes the same prediction regardless of the training set (Domingos 2000).

3.2 Addressing high variance in SGBT

Boosting model can easily over-fit to training data, since it works by building models sequentially, where each new model aims to reduce the error made by the previous one. This overfitting can result in high variance. This is more prevalent in regression problems than classification problems (Friedman et al. 2000), since the zero–one loss has a higher tolerance to variance (Domingos 2000).

Bagging is a variance-reduction technique. It allows one to smooth the variance of a learner (Bühlmann and Yu 2000). Thus in batch boosting, bagging is used to overcome high variance problem in boosting for regression (Friedman et al. 2000).

Here, simple base learners are replaced with batch bagging-based base learners for each boosting iteration. By doing so one hopes to reduce the variance in batch boosting for regression. Similar approach can be employed to reduce the high variance in the latest state-of-the-art streaming gradient boosting method SGBT for data stream regression.

This line of investigation revives and repositions a classical batch-learning idea-*using bagged base learners to reduce boosting-induced variance*-in the more demanding and dynamic setting of *streaming regression*. Despite online boosting methods being proposed as early as 2001 (Oza and Russell, 2001), to the best of our knowledge, no prior work has investigated *leveraging bagging-based streaming regressors* within a *streaming gradient-boosted framework* for *data stream regression*. This lack of adaptation forms a key research gap that our work addresses directly.

This work utilizes existing well-researched streaming bagging methods with the state-of-the-art streaming gradient boosting framework SGBT to yield two Streaming Gradient Boosted Regression (SGBR) approaches:

- Streaming Gradient Boosted Bags (SGB(Bag)): is where bagging base learners are combined in an additive setup to produce a booster. Instead of batch bagging methods here *streaming bagging* regressor Oza Bag Regressor (OzaReg) (Gomes et al. 2020) is used as the base learner for streaming gradient boosted method Sgbt. This is identified as Sgb(Oza) in rest of the paper. OzaReg in Sgb(Oza) is replaced with *streaming random subspace* method ArfReg and *streaming random patches* method Streaming Random Patches Regressor (SrpReg) (Gomes et al. 2020) to yield two additional variants of SGB(Bag): Sgb(Arf) and Sgb(Srp). Although Sgb(Soknl) is feasible, its performance in initial experiments was inferior to SOKNL. Therefore, it is excluded from this study. Nevertheless, it deserves further exploration in future research.
- Bagged Streaming Gradient Boosted Trees (Bag(Sgbt)): combines multiple Sgbt regressors in a *streaming bagging* manner using OzaReg to yeild Oza(Sgbt). This bagging setup can be further extended to yield a *streaming random patches* method with Sgbt as the base learner for SrpReg. This streaming random patches method with gradient boosted trees identified as Srp(Sgbt) in rest of the paper. Streaming subspace method ArfReg also integrates random forest techniques. Therefore, it uses a specific FIRT-DD as its base learner. Hence, Arf(Sgbt) which aggregates multiple Sgbts is not considered in this work. Soknl(Sgbt) is also not possible due to SOKNL's special base learner implementation.

In the above notation of Sgbr methods, outermost ensemble type is noted first. Figure 3 also summarizes above Sgbr variants. The mean of predictions is used as the aggregation mechanism by OzaReg, ArfReg and SrpReg. Since it produced better results than the median in the previous work (Gomes et al. 2020).

Since vanilla Sgbt allows different streaming regressors as its base learners, its time and memory complexities are influenced by the base learner's time and memory

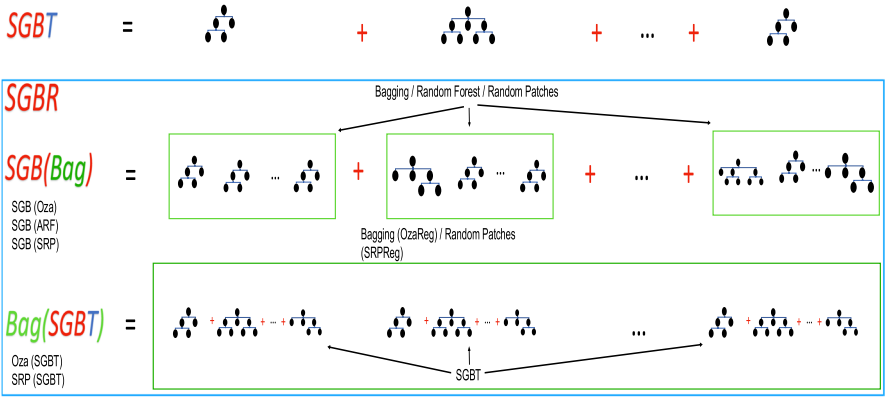


Fig. 3 Different variants of Sgbr: **a**SGB(Bag) variants: i) Sgb(Oza) uses streaming bagging OzaReg as Sgbt's base learner. ii) Sgb(Arf) uses streaming subspace ArfReg as Sgbt's base learner. iii) Sgb(Srp) uses streaming random patches SrpReg as Sgbt's base learner. **b**Bag(Sgbt) variants: i) Oza(Sgbt) aggregates multiple Sgbts via bagging OzaReg. ii) Srp(Sgbt) aggregates multiple Sgbts via random patches SrpReg. Note: OzaReg, ArfReg, and SrpReg use the mean of predictions to aggregate results from each ensemble item

complexities(Gunasekara et al. 2024). Original SGBT's time complexity is derived as $\mathcal{O}(CSf)$, and its memory complexity as $\mathcal{O}(CSf)$, assuming $\mathcal{O}(f)$ for the base learner's time and memory complexities(Gunasekara et al. 2024). There, S is the number of boosting steps, and C is the number of classes.

Vanilla SGBT handles multi-class problems either by using a committee of $C-1$ trees at each boosting iteration or by training C separate binary SGBT models for each class. However, since we focus only on regression problems, we set $C=1$. In this case, at each boosting iteration, the current streaming regression tree improves upon the cumulative prediction of the previous trees. Hence, for SGB(Bag), time and memory complexities can be derived as $\mathcal{O}(Sf_{SGBag})$ and $\mathcal{O}(Sf_{SGBag})$. Here, $\mathcal{O}(f_{SGBag})$ is the time and memory complexities of the bagging/random subspace/random patches base learner. In Bag(SGBT) case, SGBT is the base learner. Bag(SGBT)'s time and memory complexity can be derived as $\mathcal{O}(com_{Bag/SRP}(Sf))$ and $\mathcal{O}(com_{Bag/SRP}(Sf))$, where $\mathcal{O}(com_{Bag/SRP}(SGBT))$ is each bagging/random patches method's time and memory complexity functions considering their base learner SGBT.

In the experiments, we evaluate the effectiveness of SGB(Bag): SGB(OZA), SGB(ARF) and SGB(SRP) and Bag(SGBT): OZA(SGBT) and SRP(SGBT) against current state-of-the-art streaming regression methods considering both predictive power and wall time.

4 Experiments

Three sets of experiments were conducted to evaluate the effectiveness of SGBR. One compares different SGBR versions against current state-of-the-art streaming regressors. The other assesses the bias-variance decomposition of top two variants of SGBR against SGBT on different boosting iterations. Third, does a parameter exploration of the best SGBR variant.

The experiments used eleven datasets: four real-world datasets and seven synthetic datasets. Four of the synthetic datasets contain different concept drift scenarios. Below is a brief description of each dataset:

- Ailerons: is a synthetic dataset that addresses a control problem, namely flying a F16 aircraft. The attributes describe the status of the airplane, while the goal is to predict the control action on the ailerons of the aircraft.
- Elevators: is also a synthetic dataset that predicts the control problem of flying a F16 aircraft. But target variable and attributes are different from the Ailerons. In this case, the target variable relates to an action taken on the aircraft elevators.
- Fried (Friedman 1991): is a synthetic dataset that uses a highly non-linear formula to generate data. The dataset also contains five irrelevant features.
- HyperA (Hulten et al. 2001): is also a synthetic dataset with three abrupt drifts. The problem predicts the distance from a random point to a hyperplane in the space. It contains abrupt drifts at 125K, 250K, and 375K instance positions. The

window of change for abrupt drifts is one instance[1].

- FriedmanGra/Gsg/Lea: family of datasets were proposed in Ikonomovska et al. (2011). They contain different types of concept drifts. *Gra* contains global recurring abrupt drifts. *Gsg* has global slow gradual drifts, and *Lea* consists of Local expanding abrupt drifts. *Lea* has three concept drifts after 25k, 50k, and 75k instances, while the *Gra* and *Gsg* contains two concept drifts after 35k and 75k instances[2].
- Abalone (Nash et al. 1995): is a real-world dataset that aims at predicting the age of abalones based on physical measurements. The age of abalone is determined by cutting the shell through the cone, staining it, and counting the number of rings through a microscope.
- Bike (Fanaee-T and Gama 2014): dataset contains the hourly and daily count of rental bikes between the years 2011 and 2012 in the Capital bike share system with the corresponding weather and seasonal information.
- House8L: is often used to predict the US house price in 1990s. The data were collected as part of the 1990 US census. These are mostly counts cumulated at different survey levels.
- MetroTraffic: is used to predict the traffic volume from weather information.
- SUP2I (Sun et al. 2025): is a synthetic streaming dataset generated using Generative Adversarial Network (GAN)s, based on a real-world data for predicting the critical temperature of superconductors (Hamidieh 2018), as described in Sun et al. (2025). It contains 100k instances with two *incremental* drifts, with a drift width of 20k instances. The dataset is evenly divided into five segments: three stable and two drifting periods (Sun et al. 2025).
- SUP3A (Sun et al. 2025): is a synthetic streaming dataset generated using GANs trained on the same real-world data (Hamidieh 2018). It comprises four 50k concept segments, each undergoing an *abrupt* transition to the next.
- SUP3G (Sun et al. 2025): was also generated using GANs trained on the same real-world data (Hamidieh 2018). It also contains four 50k concepts, each transitioning into another within a 10k *gradual* drifting period.
- DemandF (Gunasekara et al. 2025): is a Kaggle competition dataset [39] released by one of the world's largest retail chains to develop models to predict sales for 28 Stock Keeping Units (SKUs) across 76 stores. It contains 3 years of weekly demand data. The dataset may contain seasonal recurrent drifts.
- NZEnergy (Gunasekara et al. 2025): contains energy price information in New Zealand for every half an hour roughly for two years (1/11/2022 - 15/10/2024) (Sun et al. 2024). The dataset contains seasonal recurrent drifts.

Most of the datasets are from SOKNL's empirical evaluation (Sun et al. 2022), except for the synthetic datasets with drifts FriedmanGra/Gsg/Lea, SUP/2I/3A/3 G, and the real datasets DemandF and NZEnergy. Table 2 summarizes the above information about different datasets.

[1] CapyMOA *HyperPlaneRegression* generator along with *DriftStream* is used to generate HyperA.

[2] River FriedmanDrift generator was used to generate FriedmanGra/Gsg/Lea.

Table 2 Datasets

Datasets	# features	# instances	Source
Ailerons	40	13750	Synthetic
Elevators	18	16599	Synthetic
Fried	10	40768	Synthetic
HyperA*	10	500000	Synthetic
FriedmanLea*	10	100000	Synthetic
FriedmanGra*	10	100000	Synthetic
FriedmanGsg*	10	100000	Synthetic
SUP2I*	80	100000	Synthetic
SUP3A*	81	200000	Synthetic
SUP3G*	81	200000	Synthetic
Abalone	8	4977	Real
Bike	12	17379	Real
House8L	8	22784	Real
MetroTraffic	7	48204	Real
DemandF*	8	150150	Real
NZEnergy*	12	3100660	Real

* contains drifts

Different variants of SGBR were compared against the current state-of-the-art streaming bagging method (OZAREG), the random subspaces method (ARFREG), the random subspaces with k-means method (SOKNL), the random patches method (SRPREG) and the latest gradient boosting method (AXGB). In the literature, both ARFREG and SOKNL performed best with the FIRT-DD as the base learner with *VarianceReductionSplitCriterion*, a grace period of 50 instances, split confidence of 0.01, and 100 trees (Sun et al. 2022). The MOA implementation of ARFREG and SOKNL use the above best hyperparameter configurations and other hyperparameter settings as their default parameters. Since all datasets in SOKNL empirical evaluation (Sun et al. 2022) are available in our experiments, ARFREG and SOKNL used default MOA configurations. AXGB was extended to support streaming regression using squared loss as the loss function and mean of prediction as the vote aggregation mechanism. In the original implementation, AXGB employed XGBOOST as base learners in a boosted setup (Montiel et al. 2020). To maintain consistency with other baselines that used 100 trees, AXGB was configured with 10 boosting iterations, each consisting of an XGBOOST containing 10 trees. All the other AXGB best performing configurations (learning rate of 0.3, maximum depth of 6, maximum window size of 1000, minimum window size of 1, no drift detection and 'replace' strategy) (Montiel et al. 2020) were kept unchanged.

SGBR used same FIRT-DD configurations as ARFREG and SOKNL. Also, both the SGBR setups: SGB(Bag) and Bag(SGBT) used 10 ensemble items with 10 boosting iterations so that the total number of trees in each SGBR is 100. Based on the parameter exploration done in sec 4.3, all SGBR variants employed a learning rate of 1.0 and utilized 75% of the available features. The experiments used SGB(Bag) variants: SGB(OZA), SGB(ARF), SGB(OZA) and Bag(SGBT) variants: OZA(SGBT), SRP(SGBT) for SGBR.

Furthermore, single tree regressors FIRT-DD used the same hyper-parameters as above, and HTR used the same grace period and split confidence as above. Software,

hardware configurations of the experimental systems, and further information on reproducing the results are available in Sect. 1.

Recent literature favors Coefficient of Determination (R^2) (Wright 1921), also known as R-squared, against Mean Squared Error (MSE) or Mean Absolute Error (MAE) when comparing different algorithms over multiple datasets for regression (Chicco et al. 2021). R^2 is formulated as: $R^2 = 1 - \frac{\sum_{i=1}^{n}(m(x_i)-y_i)^2}{\sum_{i=1}^{n}(y_i-\bar{y})^2}$ where $m(x_i)$ is the prediction given by model m for the instance x_i, y_i and $\bar{y}$ represents mean of all true values y_i. R^2 ranges from $-\infty$ to 1.0. For our evaluation we use Adjusted Coefficient of Determination ($\bar{R}^2$) (Yin and Fan 2001) which avoids increasing R^2 with the input dimensions p: $\bar{R}^2 = 1 - \left(1 - R^2\right)\frac{n-1}{n-p-1}$. Here, n is the sample size.

Cumulative $\bar{R}^2$ and final test-then-train wall-time (in seconds) were used for overall evaluations, while windowed $\bar{R}^2$ and the test-then-train wall-time (in seconds) at each time point were utilized for periodic plots.

4.1 Comparison against baselines

Table 3 compares different SGBR variants against streaming bagging (OZAREG), random subspaces (ARFREG), random subspaces with centroids at the leaf (SOKNL), random patches (SRPREG), vanilla streaming gradient boosted (SGBT, AXGB) methods and single tree regressors: FIRT-DD and HTR considering $\bar{R}^2$.

For SGBR, Bag(SGBT) variants: OZA(SGBT) and SRP(SGBT), and SGB(Bag) variants: SGB(OZA), SGB(ARF) and SGB(SRP) were used in the experiments.

All inner ensembles and FIRT-DD used the same FIRT-DD configurations. Single tree regressor HTR used the same split confidence and grace period as FIRT-DD. All experiments were conducted five times using different random seeds. Details regarding software and hardware configurations, along with additional information on how to reproduce the results, can be found in Sect. 1.

In Table 3 results, vanilla gradient boosting method SGBT has the worst predictive performance. But SGBR variants have the best results. There, the good performance of bagging-based method OZAREG compared to random subspace method ARFREG aligns with previous empirical results (Gomes et al. 2020).

Considering Table 3 results for SGB(OZA), SGB(ARF) SGB(SRP), OZAREG, ARFREG and SRPREG one can see that the SGB(Bag) variants improve accuracy of streaming bagging, random subspace and random patches methods. Also, SGB(OZA) seems to outperform the current state-of-the-art streaming regression method SOKNL by a considerable margin considering average rank. However, considering Table 4 wall time results, one can see that these performance gains of SGB(OZA) do not come with an extra computing overhead.

On the other hand, Bag(SGBT) variants: OZA(SGBT) and SRP(SGBT) seem to have similar predictive performance to their SGB(Bag) counterparts: SGB(OZA) and SGB(SRP) most of the time. But OZA(SGBT) and SRP(SGBT) seem to be computationally inefficient when one considers Table 4 wall time results. Per-instance multiple training iterations caused by OZAREG's and SRPREG's λ parameter on the whole SGBT boosters seem to cause the additional computing overhead compared to its effect on

Table 3 Adjusted Coefficient of Determination ($\bar{R}^2$)

method	fried	House8L	abalone	bike	elevators	MetroTraffic	ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	**avg**	**avgRank**
OZA(SGBT)	**0.89**	**0.60**	0.46	**0.93**	0.70	0.24	**0.76**	**0.88**	**0.88**	**0.88**	**0.88**	0.63	0.97	**0.58**	**0.58**	**0.58**	**0.72**	**2.2**
SRP(SGBT)	0.67	0.56	0.49	0.85	**0.73**	0.36	**0.76**	0.63	0.64	0.66	0.69	0.55	**0.98**	0.57	0.57	0.57	0.64	4.5
SGB(OZA)	**0.89**	0.58	0.48	**0.93**	0.71	0.28	0.75	**0.88**	**0.88**	0.87	0.87	**0.66**	**0.98**	0.57	**0.58**	0.57	**0.72**	2.4
SGB(ARF)	0.72	0.55	0.53	0.81	0.52	**0.51**	0.68	0.73	0.71	0.69	0.63	0.57	**0.98**	0.55	0.56	0.56	0.65	5.1
SGB(SRP)	0.67	0.55	0.48	0.79	0.51	0.31	0.66	0.63	0.60	0.60	0.49	0.60	0.97	0.54	0.56	0.55	0.59	6.9
SGBT	0.46	0.18	0.08	0.80	0.19	−0.05	0.38	0.49	0.42	0.42	0.27	0.29	0.95	0.11	0.09	0.10	0.32	11.9
AXGB	0.80	−0.53	−0.12	0.84	0.38	−0.84	0.61	0.73	0.70	0.68	0.70	−0.73	-7.7e+28	−4.16	−19.07	−22.33	-4.8e+27	9.4
OZAREG	0.82	0.54	0.37	0.82	0.53	0.15	0.62	0.81	0.79	0.79	0.82	0.49	0.81	0.55	0.57	0.57	0.63	5.8
ARFREG	0.65	0.54	0.48	0.74	0.39	0.26	0.55	0.67	0.65	0.64	0.61	0.51	0.96	0.54	0.57	0.56	0.58	7.6
SOKNL	0.82	0.58	**0.58**	0.80	0.46	0.31	0.56	0.81	0.81	0.80	0.86	0.53	0.96	0.55	**0.58**	0.57	0.66	4.4
SRPREG	0.53	0.52	0.45	0.72	0.37	0.21	0.57	0.52	0.49	0.49	0.43	0.52	0.96	0.54	0.57	0.56	0.53	9.3
FIRT-DD	0.60	0.46	0.16	0.75	0.24	0.00	0.46	0.63	0.57	0.58	0.47	0.36	0.74	0.48	0.49	0.50	0.47	10.4
HTR	0.65	0.36	0.14	0.56	0.25	0.04	0.44	0.64	0.47	0.42	0.35	0.29	0.62	0.42	0.44	0.44	0.41	11.2

The winning method for each dataset is bolded

Standard deviations are available in Table 8

a single tree setup in SGB(Bag) variants:SGB(OZA) and SGB(SRP). Unfortunately, streaming gradient boosting methods: SGBT and AXGB without any variance reduction mechanism seem to perform poorly[3].

Figure 4 presents the Shaffer post-hoc test (p-value = 0.05) for the best SGBR variants: OZA(SGBT), SGB(OZA), and top ensemble baselines: OZAREG, ARFREG, and SOKNL, based on $\bar{R}^2$. The results show no significant difference between the top SGBR variants (OZA(SGBT) and SGB(OZA)), but both significantly outperform the state-of-the-art SOKNL in terms of $\bar{R}^2$. Figure 5 shows the Shaffer post-hoc test for wall time. Here, SGB(OZA) significantly outperforms both OZA(SGBT) and SOKNL in computational efficiency. Overall, SGB(OZA) emerges as the best SGBR variant, offering strong predictive performance with lower computational overhead compared to the current state-of-the-art SOKNL.

In Fig. 6, we explore the periodic performance of SGB(OZA) against baselines SOKNL, OZAREG and ARFREG on HyperA dataset which had abrupt concept drifts occur around 125k, 250k, and 375k instances. According to the figure, compared to other methods, SGB(OZA) seems to have less impact around drift points, considering both windowed $\bar{R}^2$ and wall time. It also maintains a good predictive performance throughout the stream. In the figure, the high computing cost of SOKNL is quite evident compared to other methods.

4.2 Bias-variance decomposition

We conducted a bias-variance decomposition to further analyze the predictive performance of SGBR against SGBT. OZA(SGBT), SGB(OZA) and SGBT were considered in the experiments. Both SGBT and OZAREG for OZA(SGBT) and SGB(OZA) used FIRT-DD as the base learner with the same configurations as in the previous experiments. We utilized a similar approach to the initial FIMT-DD (Ikonomovska et al. 2011) work for bias-variance decomposition.

For datasets without drifts, the data was split into different sets using 10-fold cross-validation without shuffling. Then, bias and variance were calculated using 10 iterations of each split. For each synthetic datasets with drifts, 11 scaled-down (10%) separate datasets were generated using different random seeds. From the eleven, one dataset was left aside as the test set. Then, for a given synthetic dataset with drifts, bias, and variance were calculated using 10 iterations considering each training dataset and the test dataset. HyperA, SUP2I, SUP3A, SUP3G, DemandF and NZEnergy was excluded from this study to avoid long computing time.

For bias variance calculation, we used *mlxtend*'s $bias_variance_decomp$ function with MSE loss. Section 1 contains a detailed description of the experimental setup. The bias component represents the method's ability to model the phenomenon under study, while the variance component measures the prediction's variability with different training sets.

[3] Poor performance of AXGB also matches its classification results which failed to surpass ARF (Montiel et al. 2020). While it is interesting to explore this further, it falls outside the scope of this research. The implementation of AXGB is closely linked to XGBOOST, making it challenging to apply the variance reduction techniques discussed in this work.

Table 4 Wall Time (seconds)

method	fried	House8L	abalone	bike	elevators	MetroTraffic	ailerons	FriedManLea	FriedManGra	FriedManGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	**avg**	**avgRank**
OZA(SGBT)	58.3	27.5	4.4	14.1	20.9	33.6	22.6	172.7	173.3	173.2	1.4e+03	410.7	9.0e+03	1.1e+03	4.2e+03	4.0e+03	1.3e+03	9.9
SRP(SGBT)	83.6	39.7	7.0	29.4	46.2	79.3	55.6	310.2	353.9	365.3	2.8e+03	1.0e+03	2.2e+04	2.0e+03	6.6e+03	6.3e+03	2.6e+03	12.0
SGB(OZA)	30.8	12.9	1.9	5.1	9.0	9.9	11.0	81.3	79.7	79.6	527.4	151.9	3.2e+03	569.1	1.4e+03	1.4e+03	475.5	4.5
SGB(ARF)	38.3	24.6	3.3	11.4	11.5	20.6	15.5	108.4	111.4	109.7	805.4	365.3	6.9e+03	865.0	2.2e+03	2.1e+03	856.3	7.1
SGB(SRP)	41.4	17.4	3.8	11.7	23.2	25.5	26.3	121.1	115.5	112.0	732.4	355.8	2.3e+04	527.6	1.3e+03	1.1e+03	1.7e+03	7.5
SGBT	54.1	19.6	3.3	10.3	17.7	19.4	26.1	137.9	146.8	143.2	1.1e+03	366.4	8.2e+03	1.2e+03	3.2e+03	3.0e+03	1.1e+03	8.1
AXGB	180.7	109.4	13.0	97.6	73.4	197.5	54.7	3.7e+03	3.0e+03	3.0e+03	1.5e+04	5.9e+03	1.3e+05	3.6e+03	7.9e+03	7.1e+03	1.1e+04	12.9
OZAREG	26.6	9.9	1.6	4.7	7.2	6.7	10.5	73.4	62.9	63.4	381.1	26.3	1.2e+03	640.7	1.4e+03	1.5e+03	335.9	3.4
ARFREG	31.0	22.2	4.5	11.5	11.5	20.8	25.0	114.3	97.6	99.6	594.3	70.5	1.7e+03	1.2e+03	2.9e+03	3.0e+03	617.6	6.9
SOKNL	56.5	34.1	6.8	20.3	20.0	44.1	32.4	180.9	161.7	162.8	1.2e+03	151.2	3.9e+03	1.3e+03	3.7e+03	3.7e+03	918.0	9.8
SRPREG	31.8	13.9	3.0	11.2	21.1	19.2	30.3	87.5	92.1	86.7	484.1	78.9	4.1e+03	655.6	1.3e+03	1.2e+03	514.0	5.8
FIRT-DD	**0.6**	**0.3**	**0.1**	**0.2**	**0.2**	**0.2**	**0.3**	**1.8**	**1.8**	**1.7**	**6.2**	**0.7**	**14.5**	**13.9**	**25.9**	**24.4**	**5.8**	**1.0**
HTR	1.0	0.5	0.2	0.3	0.3	0.4	0.4	2.7	2.7	2.7	13.5	1.1	27.5	19.5	57.6	54.6	11.6	2.0

The winning method for each dataset is bolded

Standard deviations are available in Table 9

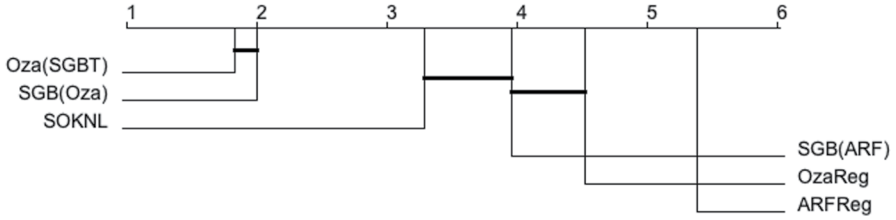


Fig. 4 Shaffer Post-hoc test with p-value 0.05 for best SGBR variants and best baselines considering $\bar{R}^2$. **Lower is better**

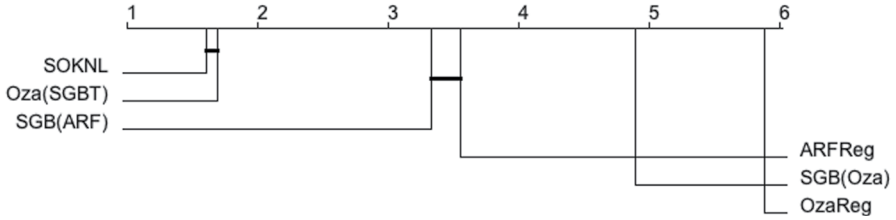


Fig. 5 Shaffer Post-hoc test with p-value 0.05 considering Wall Time for the selected methods the in Fig. 4. **Higher is better**

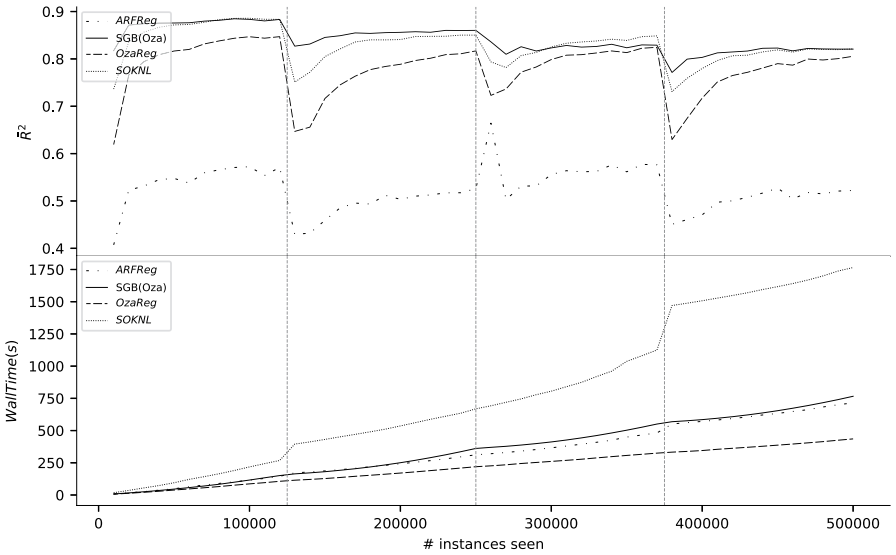


Fig. 6 Over-time windowed (10k), $\bar{R}^2$ and wall time (seconds) for SGB(OZA), ARFREGARFREG and SOKNL on **HyperA**. The vertical dotted lines mark the start of a concept. Abrupt concept drifts occur around 125k, 250k, and 375k instances. Ensemble configurations: SGB(OZA) used 10 boosting iterations and 10 base learners for OZAREG. All other baselines each used 100 base learners

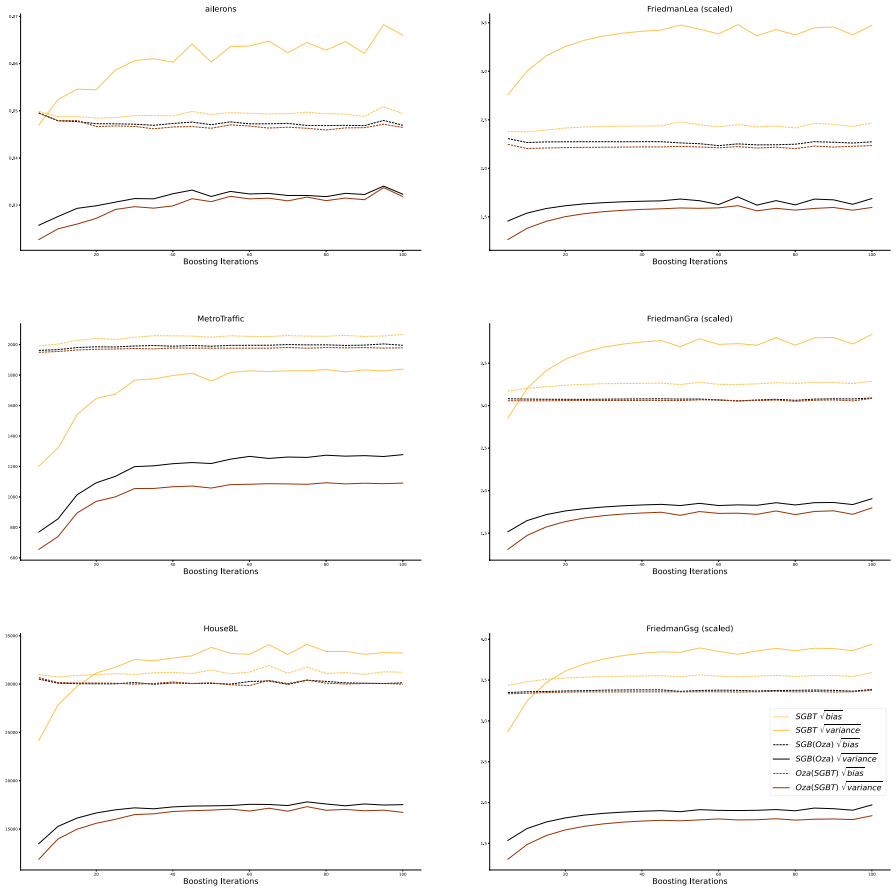


Fig. 7 Bias (–) variance (-) decomposition of SGBT [yellow], SGB(OZA) [black] and OZA(SGBT) [brown], for different boosting iterations

As shown in Figs. 7 and 8, variance for SGBT increases with boosting iterations while keeping bias pretty much unchanged. On the other hand, for SGB(OZA), variance is comparatively low with pretty much the similar bias as SGBT. This comparatively low bias is caused by bagging base learners (Friedman et al. 2000). Furthermore, if one configures the ensemble in Bag(SGBT) way: OZA(SGBT), where several SGBTs are aggregated using a streaming bagging regressor, variance further decreases compared to SGB(OZA), where bagging ensembles are base learners for SGBT. This result also aligns with Table 3 results where OZA(SGBT) has a slightly better ranking compared to SGB(OZA). However, as discussed above, this performance gain comes with a computing cost.

4.3 Parameter exploration

To investigate the impact of learning rate and feature percentage, we conducted a parameter search by varying the learning rate values (0.5, 1.0, 1.5) and feature

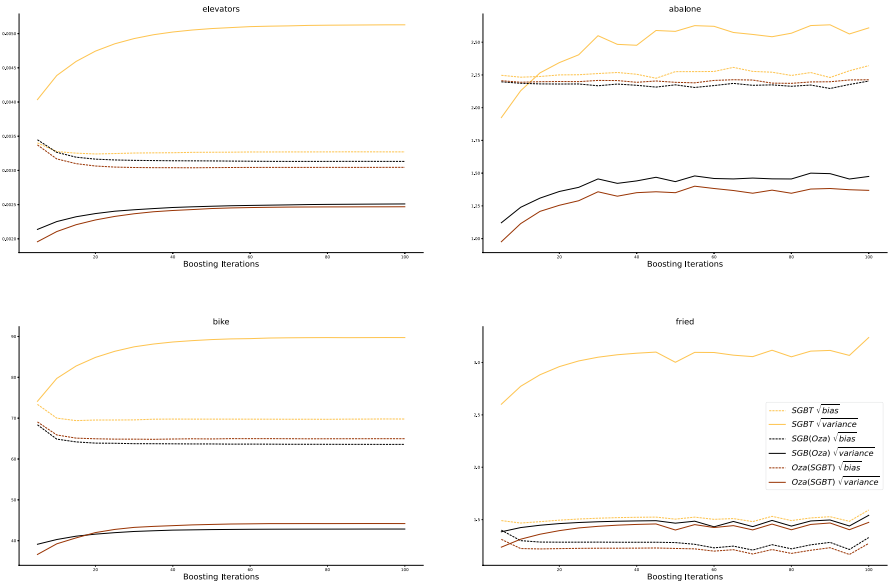


Fig. 8 Bias (–) variance (-) decomposition of SGBT [yellow], SGB(OZA) [black] and OZA(SGBT) [brown], for different boosting iterations

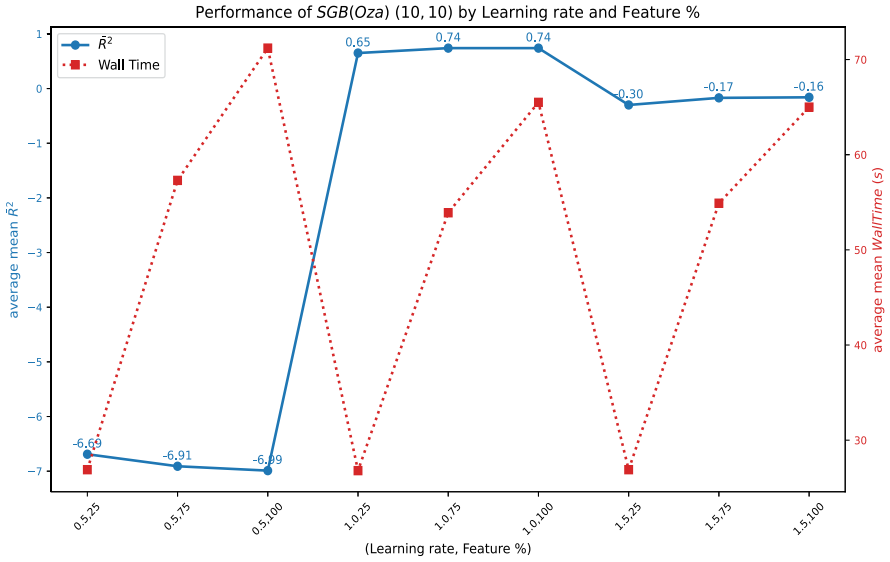


Fig. 9 Effect of learning rate and feature % on SGB(OZA) (10,10) (10 boosting iteration, 10 bagging ensemble size := 100 trees) for the selected datasets (excluding SUP2I, SUP3A, SUP3G, DemandF and NZEnergy) considering **average mean** $\bar{R}^2$ and Wall Time (seconds)

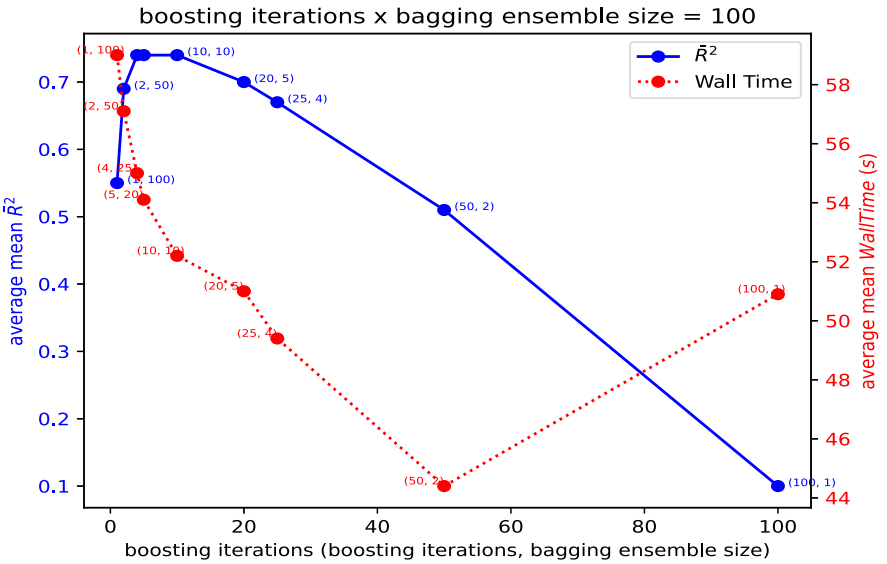


Fig. 10 Effect of boosting steps and bagging ensemble size on SGB(OZA) (100 trees, learning rate of 1.0 and 75% of the available features) for the selected datasets (excluding SUP2I, SUP3A, SUP3G, DemandF and NZEnergy) considering **average mean** $\bar{R}^2$ and Wall Time (seconds)

percentages $(25\%, 75\%, 100\%)$, while keeping the boosting iterations and bagging ensemble size fixed at (10, 10) for SGB(OZA) across selected datasets (excluding SUP2I, SUP3A, SUP3G, DemandF and NZEnergy). Each configuration was executed five times with different random seeds. The Sect. 1 contains information regarding the software and hardware setups, along with instructions on how to replicate the results. The corresponding results are presented in Fig. 9.

Based on the mean $\bar{R}^2$ values, a learning rate 1.0 consistently yields strong performance across datasets. Notably, combining a learning rate of 1.0 with 75% of the available features appears to strike the best balance between predictive accuracy and computational efficiency. Increasing the proportion of features generally leads to higher wall times, which aligns with established knowledge in regression tree learning, where larger feature percentages tend to produce deeper and more complex trees, increasing computation time.

Further experiments were conducted to understand the effect of boosting iterations and the bagging ensemble size on SGB(OZA) keeping the total number of trees to 100. In the experiments the base learner configurations were kept unchanged. All SGB(OZA) variants used a learning rate of 1.0 and 75% of the available features. Each method was run 5 times on each dataset.

Figure 10 along with Tables 5 and 6 show the effect of boosting iterations and ensemble size on SGB(OZA)'s mean $\bar{R}^2$ and mean wall time. Since we keep the number of trees to 100, each annotation in Fig. 10 and each row in Tables 5 and 6 show a boosting iteration, bagging ensemble size combination. When one increases the number of boosting iterations, the bagging ensemble size decreases. From the Fig. 10, one can see that (10, 10) combination (10 boosting iterations with 10 bagging ensemble

Table 5 Effect of boosting steps and bagging ensemble size on SGB(OZA) (100 trees, learning rate of 1.0 and 75% of the available features) for each dataset

(boosting iter, bagging size)	fried	House8L	abalone	bike	elevators	MetroTraffic	ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	**avg**	**avgRank**
1,100	0.63	0.53	0.36	0.74	0.50	0.14	0.52	0.67	0.62	0.62	0.69	0.55	7.1
2,50	0.88	0.57	0.41	0.89	0.61	0.21	0.67	0.87	0.86	0.83	0.84	0.69	4.9
4,25	**0.91**	**0.60**	0.43	0.92	0.68	0.25	0.74	**0.90**	0.89	0.88	**0.91**	**0.74**	2.6
5,20	**0.91**	**0.60**	0.45	**0.93**	0.70	0.26	**0.75**	**0.90**	**0.90**	**0.89**	**0.91**	**0.74**	**2.1**
10,10	0.89	0.58	0.48	**0.93**	**0.71**	0.28	**0.75**	0.88	0.88	0.87	0.87	**0.74**	2.4
20,5	0.84	0.52	**0.49**	0.92	0.66	**0.30**	0.73	0.84	0.83	0.82	0.79	0.70	4.1
25,4	0.81	0.49	0.45	0.91	0.64	0.29	0.70	0.81	0.80	0.79	0.74	0.67	5.2
50,2	0.63	0.33	0.27	0.86	0.46	0.15	0.57	0.64	0.62	0.60	0.47	0.51	7.7
100,1	0.18	0.03	−0.09	0.74	−0.08	−0.23	0.23	0.21	0.14	0.14	−0.21	0.10	9.0

The winning method for each dataset is bolded

Standard deviations are available in Table 10

Adjusted coefficient of determination ($\bar{R}^2$)

Table 6 Effect of boosting steps and bagging ensemble size on SGB(OZA) (100 trees, learning rate of 1.0 and 75% of the available features) for each dataset. Wall Time (seconds)

(Boosting iter, bagging size)	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	avg	avgRank
1,100	27.3	9.9	1.7	4.1	**6.8**	8.5	**8.2**	97.4	64.7	71.6	349.3	59.0	5.5
2,50	27.5	**9.6**	1.5	3.8	7.0	8.2	8.5	89.2	64.6	68.3	339.7	57.1	4.5
4,25	26.0	9.7	1.5	**3.6**	7.0	7.4	9.1	75.9	**64.0**	66.5	334.3	55.0	3.9
5,20	25.6	**9.6**	**1.4**	**3.6**	7.1	**7.0**	9.2	72.3	64.1	65.8	328.8	54.1	**3.1**
10,10	24.9	9.9	1.5	3.8	7.4	**7.0**	10.0	66.6	64.5	**64.7**	314.1	52.2	3.5
20,5	24.5	10.3	1.7	4.3	8.4	8.0	11.0	64.4	65.9	66.2	296.3	51.0	4.8
25,4	**23.6**	10.2	1.7	4.5	8.7	8.6	11.5	63.0	66.5	66.4	278.5	49.4	5.3
50,2	25.2	10.6	2.0	5.6	10.2	10.8	14.4	**62.8**	67.6	68.8	**210.0**	**44.4**	6.3
100,1	30.6	13.1	2.6	7.6	13.6	14.7	20.5	75.9	77.4	76.7	226.8	50.9	8.1

The winning method for each dataset is bolded

Standard deviations are available in Table 11

Table 7 $\bar{R}^2$ and Wall Time (seconds) for **best**SGB(OZA) configurations (boosting steps, bagging ensemble size) with learning rate of 1.0 and 75% of the available features **against** best performing baselines: SOKNL, OZAREG and ARFREG on each dataset. 100 trees for each method

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	avg	avgRank
$\bar{R}^2$													
SGB(OZA)(5,20)	**0.91**	**0.60**	0.45	**0.93**	0.70	0.26	**0.75**	**0.90**	**0.90**	**0.89**	**0.91**	**0.74**	**1.8**
SGB(OZA)(10,10)	0.89	0.58	0.48	**0.93**	**0.71**	0.28	**0.75**	0.88	0.88	0.87	0.87	**0.74**	**1.8**
SOKNL	0.82	0.58	**0.58**	0.80	0.46	**0.31**	0.56	0.81	0.81	0.80	0.86	0.67	3.1
OZAREG	0.82	0.54	0.37	0.82	0.53	0.15	0.62	0.81	0.79	0.79	0.81	0.64	3.7
ARFREG	0.65	0.54	0.48	0.74	0.39	0.26	0.55	0.67	0.65	0.64	0.58	0.56	4.6
Wall Time (seconds)													
SGB(OZA)(5,20)	25.6	**9.6**	**1.4**	**3.6**	7.1	7.0	**9.2**	72.3	64.1	65.8	328.8	54.1	2.0
SGB(OZA)(10,10)	**24.9**	9.9	1.5	3.8	7.4	7.0	10.0	**66.6**	64.5	64.7	314.1	52.2	2.1
SOKNL	51.0	32.8	6.6	18.6	19.4	43.9	31.3	188.5	160.1	166.9	739.8	132.6	5.0
OZAREG	26.5	9.9	**1.4**	4.4	**6.8**	**6.2**	10.4	77.4	**58.9**	**59.3**	**265.2**	**47.9**	**1.9**
ARFREG	29.9	22.0	4.0	10.1	10.2	17.6	23.4	163.2	92.5	124.3	340.8	76.2	4.0

The winning method for each dataset is bolded

Standard deviations are available in Table 12

Table 8 Adjusted Coefficient of Determination ($\bar{R}^2$)

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	avg	avgRank
OZA(SGBT)	**0.89** ± 0.00	**0.60** ± 0.00	0.46 ± 0.02	**0.93** ± 0.00	0.70 ± 0.01	0.24 ± 0.00	**0.76** ± 0.01	**0.88** ± 0.00	**0.88** ± 0.00	**0.88** ± 0.00	**0.88** ± 0.00	0.63 ± 0.00	0.97 ± 0.00	**0.58** ± 0.00	**0.58** ± 0.00	**0.58** ± 0.00	**0.72**	**2.2**
SRP(SGBT)	0.67 ± 0.06	0.56 ± 0.01	0.49 ± 0.02	0.85 ± 0.02	**0.73** ± 0.02	0.36 ± 0.02	0.76 ± 0.01	0.63 ± 0.04	0.64 ± 0.05	0.66 ± 0.04	0.69 ± 0.01	0.55 ± 0.00	**0.98** ± 0.00	0.57 ± 0.00	0.57 ± 0.00	0.57 ± 0.00	0.64	4.5
SGB(OZA)	0.89 ± 0.00	0.58 ± 0.00	0.48 ± 0.01	0.93 ± 0.01	0.71 ± 0.01	0.28 ± 0.01	0.75 ± 0.02	0.88 ± 0.00	0.88 ± 0.00	0.87 ± 0.01	0.87 ± 0.00	**0.66** ± 0.00	0.98 ± 0.00	0.57 ± 0.00	0.58 ± 0.00	0.57 ± 0.00	0.72	2.4
SGB(ARF)	0.72 ± 0.01	0.55 ± 0.01	0.53 ± 0.01	0.81 ± 0.03	0.52 ± 0.02	**0.51** ± 0.01	0.68 ± 0.01	0.73 ± 0.02	0.71 ± 0.02	0.69 ± 0.02	0.63 ± 0.01	0.57 ± 0.01	0.98 ± 0.01	0.55 ± 0.00	0.56 ± 0.00	0.56 ± 0.00	0.65	5.1
SGB(SRP)	0.67 ± 0.02	0.55 ± 0.00	0.48 ± 0.02	0.79 ± 0.04	0.51 ± 0.01	0.31 ± 0.00	0.66 ± 0.02	0.63 ± 0.02	0.60 ± 0.03	0.60 ± 0.04	0.49 ± 0.03	0.60 ± 0.01	0.97 ± 0.00	0.54 ± 0.00	0.56 ± 0.00	0.55 ± 0.00	0.59	6.9
SGBT	0.46 ± 0.02	0.18 ± 0.02	0.08 ± 0.02	0.80 ± 0.03	0.19 ± 0.04	−0.05 ± 0.02	0.38 ± 0.07	0.49 ± 0.03	0.42 ± 0.05	0.42 ± 0.03	0.27 ± 0.01	0.29 ± 0.02	0.95 ± 0.01	0.11 ± 0.01	0.09 ± 0.02	0.10 ± 0.00	0.32	11.9
AXGB	0.80 ± 0.00	−0.53 ± 0.00	−0.12 ± 0.00	0.84 ± 0.00	0.38 ± 0.00	−0.84 ± 0.00	0.61 ± 0.00	0.73 ± 0.00	0.70 ± 0.00	0.68 ± 0.00	0.70 ± 0.00	−0.73 ± 0.00	-7.7e+28 ± 0.00	−4.16 ± 0.00	−19.07 ± 0.00	−22.33 ± 0.00	-4.8e+27	9.4
OZAREG	0.82 ± 0.00	0.54 ± 0.00	0.37 ± 0.00	0.82 ± 0.00	0.53 ± 0.00	0.15 ± 0.00	0.62 ± 0.00	0.81 ± 0.00	0.79 ± 0.00	0.79 ± 0.00	0.82 ± 0.00	0.49 ± 0.00	0.81 ± 0.00	0.55 ± 0.00	0.57 ± 0.00	0.57 ± 0.00	0.63	5.8
ARFREG	0.65 ± 0.00	0.54 ± 0.00	0.48 ± 0.00	0.74 ± 0.00	0.39 ± 0.01	0.26 ± 0.00	0.55 ± 0.00	0.67 ± 0.00	0.65 ± 0.00	0.64 ± 0.00	0.61 ± 0.00	0.51 ± 0.00	0.96 ± 0.00	0.54 ± 0.00	0.57 ± 0.00	0.56 ± 0.00	0.58	7.6

Table 8 (continued)

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	avg	avgRank
SOKNL	0.82 ± 0.00	0.58 ± 0.00	**0.58** ± 0.00	0.80 ± 0.00	0.46 ± 0.01	0.31 ± 0.00	0.56 ± 0.00	0.81 ± 0.00	0.81 ± 0.00	0.80 ± 0.00	0.86 ± 0.00	0.53 ± 0.00	0.96 ± 0.00	0.55 ± 0.00	0.58 ± 0.00	0.57 ± 0.00	0.66	4.4
SRPREG	0.53 ± 0.01	0.52 ± 0.00	0.45 ± 0.00	0.72 ± 0.01	0.37 ± 0.00	0.21 ± 0.00	0.57 ± 0.00	0.52 ± 0.01	0.49 ± 0.01	0.49 ± 0.01	0.43 ± 0.00	0.52 ± 0.00	0.96 ± 0.00	0.54 ± 0.00	0.57 ± 0.00	0.56 ± 0.00	0.53	9.3
FIRT-DD	0.60 ± 0.00	0.46 ± 0.00	0.16 ± 0.00	0.75 ± 0.00	0.24 ± 0.00	−0.00 ± 0.00	0.46 ± 0.00	0.63 ± 0.00	0.57 ± 0.00	0.58 ± 0.00	0.47 ± 0.00	0.36 ± 0.00	0.74 ± 0.00	0.48 ± 0.00	0.49 ± 0.00	0.50 ± 0.00	0.47	10.4
HTR	0.65 ± 0.00	0.36 ± 0.00	0.14 ± 0.00	0.56 ± 0.00	0.25 ± 0.00	0.04 ± 0.00	0.44 ± 0.00	0.64 ± 0.00	0.47 ± 0.00	0.42 ± 0.00	0.35 ± 0.00	0.29 ± 0.00	0.62 ± 0.00	0.42 ± 0.00	0.44 ± 0.00	0.44 ± 0.00	0.41	11.2

Standard deviations are rounded to two decimal places, which can result in some values being displayed as 0.00 in certain cases

Table 9 Wall Time (seconds)

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedManLea	FriedManGra	FriedManGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	avg	avgRank
OZA(SGBT)	58.3 ± 1.6	27.5 ± 6.7	4.4 ± 0.3	14.1 ± 0.7	20.9 ± 1.0	33.6 ± 1.2	22.6 ± 1.1	172.7 ± 45.2	173.3 ± 44.5	173.2 ± 45.9	1.4e+03 ± 87.8	410.7 ± 34.3	9.0e+03 ± 363.0	1.1e+03 ± 100.1	4.2e+03 ± 740.6	4.0e+03 ± 736.1	1.3e+03	9.9
SRP(SGBT)	83.6 ± 8.7	39.7 ± 9.3	7.0 ± 0.4	29.4 ± 2.1	46.2 ± 3.3	79.3 ± 5.2	55.6 ± 3.2	310.2 ± 88.6	353.9 ± 100.0	365.3 ± 112.5	2.8e+03 ± 210.1	1.0e+03 ± 62.0	2.2e+04 ± 6.0e+03	2.0e+03 ± 180.7	6.6e+03 ± 1.1e+03	6.3e+03 ± 1.0e+03	2.6e+03	12.0
SGB(OZA)	30.8 ± 5.5	12.9 ± 2.9	1.9 ± 0.2	5.1 ± 0.3	9.0 ± 0.7	9.9 ± 0.5	11.0 ± 0.6	81.3 ± 17.1	79.7 ± 14.2	79.6 ± 14.7	527.4 ± 52.8	151.9 ± 19.5	3.2e+03 ± 242.6	569.1 ± 64.5	1.4e+03 ± 48.1	1.4e+03 ± 30.3	475.5	4.5
SGB(ARF)	38.3 ± 6.9	24.6 ± 4.7	3.3 ± 0.3	11.4 ± 0.9	11.5 ± 0.5	20.6 ± 1.6	15.5 ± 1.1	108.4 ± 17.1	111.4 ± 19.0	109.7 ± 19.2	805.4 ± 46.9	365.3 ± 38.2	6.9e+03 ± 500.9	865.0 ± 64.9	2.2e+03 ± 169.0	2.1e+03 ± 191.3	856.3	7.1
SGB(SRP)	41.4 ± 7.6	17.4 ± 4.2	3.8 ± 0.5	11.7 ± 0.5	23.2 ± 2.8	25.5 ± 1.4	26.3 ± 1.2	121.1 ± 22.6	115.5 ± 22.0	112.0 ± 19.7	732.4 ± 129.1	355.8 ± 24.8	2.3e+04 ± 9.8e+03	527.6 ± 45.9	1.3e+03 ± 244.2	1.1e+03 ± 178.6	1.7e+03	7.5
SGBT	54.1 ± 7.5	19.6 ± 0.3	3.3 ± 0.2	10.3 ± 0.6	17.7 ± 0.7	19.4 ± 0.9	26.1 ± 1.7	137.9 ± 19.8	146.8 ± 18.8	143.2 ± 21.9	1.1e+03 ± 174.4	366.4 ± 20.6	8.2e+03 ± 690.9	1.2e+03 ± 91.0	3.2e+03 ± 257.9	3.0e+03 ± 326.6	1.1e+03	8.1
AXGB	180.7 ± 137.9	109.4 ± 74.2	13.0 ± 8.7	97.6 ± 83.4	73.4 ± 50.1	197.5 ± 141.7	54.7 ± 40.4	3.7e+03 ± 5.1e+03	3.0e+03 ± 5.2e+03	3.0e+03 ± 5.2e+03	1.5e+04 ± 2.7e+04	5.9e+03 ± 5.1e+03	1.3e+05 ± 1.2e+05	3.6e+03 ± 4.7e+03	7.9e+03 ± 9.3e+03	7.1e+03 ± 9.6e+03	1.1e+04	12.9
OZAREG	26.6 ± 2.8	9.9 ± 0.3	1.6 ± 0.1	4.7 ± 0.4	7.2 ± 0.2	6.7 ± 1.0	10.5 ± 0.6	73.4 ± 8.4	62.9 ± 5.1	63.4 ± 5.3	381.1 ± 40.0	26.3 ± 4.4	1.2e+03 ± 104.0	640.7 ± 46.8	1.4e+03 ± 37.0	1.5e+03 ± 89.5	335.9	3.4
ARFREG	31.0 ± 4.0	22.2 ± 0.5	4.5 ± 0.4	11.5 ± 0.4	11.5 ± 0.1	20.8 ± 1.6	25.0 ± 2.2	114.3 ± 10.3	97.6 ± 9.8	99.6 ± 10.0	594.3 ± 62.7	70.5 ± 11.4	1.7e+03 ± 178.2	1.2e+03 ± 127.0	2.9e+03 ± 142.1	3.0e+03 ± 307.3	617.6	6.9

Table 9 (continued)

Method	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedManLea	FriedManGra	FriedManGsg	hyperA	DemandF	NZEnergy	SUP2I	SUP3A	SUP3G	avg	avgRank
SOKNL	56.5 ± 7.2	34.1 ± 2.8	6.8 ± 0.4	20.3 ± 1.6	20.0 ± 1.6	44.1 ± 1.3	32.4 ± 3.5	180.9 ± 24.7	161.7 ± 20.5	162.8 ± 17.9	1.2e+03 ± 172.1	151.2 ± 25.9	3.9e+03 ± 449.0	1.3e+03 ± 125.3	3.7e+03 ± 175.2	3.7e+03 ± 326.1	918.0	9.8
SRPREG	31.8 ± 4.3	13.9 ± 0.4	3.0 ± 0.3	11.2 ± 0.7	21.1 ± 0.6	19.2 ± 0.5	30.3 ± 2.6	87.5 ± 11.4	92.1 ± 10.6	86.7 ± 11.1	484.1 ± 50.7	78.9 ± 7.2	4.1e+03 ± 768.7	655.6 ± 55.9	1.3e+03 ± 58.0	1.2e+03 ± 93.2	514.0	5.8
FIRT-DD	**0.6** ± 0.2	**0.3** ± 0.0	**0.1** ± 0.0	**0.2** ± 0.0	**0.2** ± 0.0	**0.2** ± 0.0	**0.3** ± 0.0	**1.8** ± 0.4	**1.8** ± 0.4	**1.7** ± 0.4	**6.2** ± 1.0	**0.7** ± 0.2	**14.5** ± 1.8	**13.9** ± 0.9	**25.9** ± 1.6	**24.4** ± 2.1	**5.8**	**1.0**
HTR	1.0 ± 0.3	0.5 ± 0.1	0.2 ± 0.1	0.3 ± 0.0	0.3 ± 0.0	0.4 ± 0.0	0.4 ± 0.0	2.7 ± 0.7	2.7 ± 0.7	2.7 ± 0.6	13.5 ± 2.2	1.1 ± 0.3	27.5 ± 3.8	19.5 ± 1.6	57.6 ± 4.1	54.6 ± 4.2	11.6	2.0

Standard deviations are rounded to one decimal place, which can result in some values being displayed as 0.0 in certain cases

Table 10 Effect of boosting steps and bagging ensemble size on SGB(OZA) (100 trees) for each dataset

(Boosting iter, bagging size)	Fried	House8L	Abalone	Bike	elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	avg	avgRank
1,100	0.63 ± 0.13	0.53 ± 0.01	0.36 ± 0.01	0.74 ± 0.22	0.50 ± 0.06	0.14 ± 0.01	0.52 ± 0.13	0.67 ± 0.10	0.62 ± 0.11	0.62 ± 0.03	0.69 ± 0.04	0.55	7.1
2,50	0.88 ± 0.03	0.57 ± 0.02	0.41 ± 0.01	0.89 ± 0.02	0.61 ± 0.01	0.21 ± 0.00	0.67 ± 0.02	0.87 ± 0.03	0.86 ± 0.02	0.83 ± 0.07	0.84 ± 0.06	0.69	4.9
4,25	**0.91** ± 0.01	**0.60** ± 0.01	0.43 ± 0.01	0.92 ± 0.01	0.68 ± 0.00	0.25 ± 0.02	0.74 ± 0.01	**0.90** ± 0.01	0.89 ± 0.01	0.88 ± 0.01	**0.91** ± 0.01	**0.74**	2.5
5,20	0.91 ± 0.01	0.60 ± 0.00	0.45 ± 0.02	**0.93** ± 0.01	0.70 ± 0.01	0.26 ± 0.01	**0.75** ± 0.01	0.90 ± 0.01	**0.90** ± 0.00	**0.89** ± 0.01	0.91 ± 0.00	0.74	**2.1**
10,10	0.89 ± 0.00	0.58 ± 0.00	0.48 ± 0.01	0.93 ± 0.01	**0.71** ± 0.01	0.28 ± 0.01	0.75 ± 0.02	0.88 ± 0.00	0.88 ± 0.00	0.87 ± 0.01	0.87 ± 0.01	0.74	2.4
20,5	0.84 ± 0.00	0.52 ± 0.00	**0.49** ± 0.01	0.92 ± 0.01	0.66 ± 0.01	**0.30** ± 0.00	0.73 ± 0.01	0.84 ± 0.00	0.83 ± 0.00	0.82 ± 0.01	0.79 ± 0.01	0.70	4.1
25,4	0.81 ± 0.01	0.49 ± 0.01	0.45 ± 0.01	0.91 ± 0.01	0.64 ± 0.01	0.29 ± 0.00	0.70 ± 0.01	0.81 ± 0.00	0.80 ± 0.00	0.79 ± 0.01	0.74 ± 0.02	0.67	5.2
50,2	0.63 ± 0.02	0.33 ± 0.01	0.27 ± 0.01	0.86 ± 0.02	0.46 ± 0.02	0.15 ± 0.02	0.57 ± 0.03	0.64 ± 0.02	0.62 ± 0.02	0.60 ± 0.02	0.47 ± 0.05	0.51	7.7
100,1	0.18 ± 0.07	0.03 ± 0.03	−0.09 ± 0.01	0.74 ± 0.04	−0.08 ± 0.03	−0.23 ± 0.05	0.23 ± 0.11	0.21 ± 0.06	0.14 ± 0.07	0.14 ± 0.05	−0.21 ± 0.12	0.10	9.0

Adjusted coefficient of determination ($\bar{R}^2$)

Standard deviations are rounded to two decimal places, which can result in some values being displayed as 0.00 in certain cases

Table 11 Effect of boosting steps and bagging ensemble size on SGB(OZA) (100 trees) for each dataset

(Boosting iter, bagging size)	Fried	House8L	Abalone	Bike	Elevators	MetroTraffic	Ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	avg	avgRank
1100	27.3 ± 1.4	9.9 ± 1.6	1.7 ± 0.2	4.1 ± 0.9	**6.8** ± 0.6	8.5 ± 0.9	**8.2** ± 1.7	97.4 ± 0.9	64.7 ± 1.0	71.6 ± 6.1	349.3 ± 181.0	59.0	5.5
2,50	27.5 ± 1.5	**9.6** ± 1.0	1.5 ± 0.1	3.8 ± 0.5	7.0 ± 0.8	8.2 ± 0.7	8.5 ± 1.2	89.2 ± 4.6	64.6 ± 0.6	68.3 ± 2.5	339.7 ± 178.8	57.1	4.5
4,25	26.0 ± 1.3	9.7 ± 0.9	1.5 ± 0.2	**3.6** ± 0.3	7.0 ± 0.5	7.4 ± 0.9	9.1 ± 1.0	75.9 ± 2.1	**64.0** ± 0.6	66.5 ± 1.1	334.3 ± 179.2	55.0	3.9
5,20	25.6 ± 0.9	9.6 ± 0.6	**1.4** ± 0.1	3.6 ± 0.3	7.1 ± 0.5	**7.0** ± 0.6	9.2 ± 1.0	72.3 ± 1.8	64.1 ± 0.2	65.8 ± 0.8	328.8 ± 179.4	54.1	**3.1**
10,10	24.9 ± 1.4	9.9 ± 0.7	1.5 ± 0.1	3.8 ± 0.2	7.4 ± 0.5	7.0 ± 0.2	10.0 ± 0.5	66.6 ± 1.1	64.5 ± 0.2	**64.7** ± 0.6	314.1 ± 175.8	52.2	3.5
20,5	24.5 ± 1.6	10.3 ± 0.8	1.7 ± 0.1	4.3 ± 0.2	8.4 ± 0.8	8.0 ± 0.3	11.0 ± 0.4	64.4 ± 1.1	65.9 ± 0.4	66.2 ± 0.4	296.3 ± 171.9	51.0	4.8
25,4	**23.6** ± 1.3	10.2 ± 0.8	1.7 ± 0.1	4.5 ± 0.2	8.7 ± 1.0	8.6 ± 0.3	11.5 ± 0.5	63.0 ± 0.4	66.5 ± 0.7	66.4 ± 0.4	278.5 ± 159.6	49.4	5.3
50,2	25.2 ± 1.1	10.6 ± 0.6	2.0 ± 0.1	5.6 ± 0.3	10.2 ± 0.7	10.8 ± 0.3	14.4 ± 0.7	**62.8** ± 0.5	67.6 ± 0.7	68.8 ± 2.0	**210.0** ± 106.6	**44.4**	6.3
100,1	30.6 ± 1.1	13.1 ± 0.7	2.6 ± 0.1	7.6 ± 0.4	13.6 ± 0.8	14.7 ± 0.7	20.5 ± 1.1	75.9 ± 0.5	77.4 ± 1.3	76.7 ± 1.2	226.8 ± 109.5	50.9	8.1

Wall Time (seconds)

Standard deviations are rounded to one decimal place, which can result in some values being displayed as 0.0 in certain cases

Table 12 $\bar{R}^2$ and Wall Time (seconds) for **best**SGB(OZA) configurations (boosting steps, bagging ensemble size) **against** best performing baselines: SOKNL, OZAREG and ARFREG on each dataset

method	fried	House8L	abalone	bike	elevators	MetroTraffic	ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	**avg**	**avgRank**
$\bar{R}^2$													
SGB(OZA)(5,20)	**0.91** ± 0.01	**0.60** ± 0.00	0.45 ± 0.02	**0.93** ± 0.01	0.70 ± 0.01	0.26 ± 0.01	**0.75** ± 0.01	**0.90** ± 0.01	**0.90** ± 0.00	**0.89** ± 0.01	**0.91** ± 0.00	**0.74**	**1.8**
SGB(OZA)(10,10)	0.89 ± 0.00	0.58 ± 0.00	0.48 ± 0.01	0.93 ± 0.01	**0.71** ± 0.01	0.28 ± 0.01	0.75 ± 0.02	0.88 ± 0.00	0.88 ± 0.00	0.87 ± 0.01	0.87 ± 0.01	0.74	1.8
SOKNL	0.82 ± 0.00	0.58 ± 0.00	**0.58** ± 0.00	0.80 ± 0.00	0.46 ± 0.01	**0.31** ± 0.00	0.56 ± 0.00	0.81 ± 0.00	0.81 ± 0.00	0.80 ± 0.00	0.86 ± 0.01	0.67	3.1
OZAREG	0.82 ± 0.00	0.54 ± 0.00	0.37 ± 0.00	0.82 ± 0.00	0.53 ± 0.00	0.15 ± 0.00	0.62 ± 0.00	0.81 ± 0.00	0.79 ± 0.00	0.79 ± 0.00	0.81 ± 0.01	0.64	3.7
ARFREG	0.65 ± 0.00	0.54 ± 0.00	0.48 ± 0.00	0.74 ± 0.00	0.39 ± 0.01	0.26 ± 0.00	0.55 ± 0.00	0.67 ± 0.00	0.65 ± 0.00	0.64 ± 0.00	0.58 ± 0.04	0.56	4.5
Wall Time (seconds)													
SGB(OZA)(5,20)	25.6 ± 0.9	**9.6** ± 0.6	**1.4** ± 0.1	**3.6** ± 0.3	7.1 ± 0.5	7.0 ± 0.6	**9.2** ± 1.0	72.3 ± 1.8	64.1 ± 0.2	65.8 ± 0.8	328.8 ± 179.4	54.1	2.0
SGB(OZA)(10,10)	**24.9** ± 1.4	9.9 ± 0.7	1.5 ± 0.1	3.8 ± 0.2	7.4 ± 0.5	7.0 ± 0.2	10.0 ± 0.5	**66.6** ± 1.1	64.5 ± 0.2	64.7 ± 0.6	314.1 ± 175.8	52.2	2.1
SOKNL	51.0 ± 1.5	32.8 ± 0.8	6.6 ± 0.2	18.6 ± 0.2	19.4 ± 0.7	43.9 ± 2.2	31.3 ± 0.6	188.5 ± 5.8	160.1 ± 7.0	166.9 ± 27.3	739.8 ± 473.6	132.6	5.0

Table 12 (continued)

method	fried	House8L	abalone	bike	elevators	MetroTraffic	ailerons	FriedmanLea	FriedmanGra	FriedmanGsg	hyperA	**avg**	**avgRank**
OzaReg	26.5 ± 0.5	9.9 ± 0.4	1.4 ± 0.0	4.4 ± 0.0	**6.8** ± 0.0	**6.2** ± 0.0	10.4 ± 0.2	77.4 ± 3.9	**58.9** ± 1.3	**59.3** ± 0.4	**265.2** ± 127.1	**47.9**	**1.9**
ArfReg	29.9 ± 1.3	22.0 ± 0.8	4.0 ± 0.1	10.1 ± 0.2	10.2 ± 0.2	17.6 ± 0.1	23.4 ± 0.2	163.2 ± 46.0	92.5 ± 4.6	124.3 ± 44.2	340.8 ± 189.4	76.2	4.0

100 trees for each method

$\bar{R}^2$ standard deviations are rounded to two decimal places, which can result in some values being displayed as 0.00 in certain cases

Wall time standard deviations are rounded to one decimal place, which can result in some values being displayed as 0.0 in certain cases

size) seems to give best performance considering both average mean $\bar{R}^2$ and wall time on the selected datasets. However, when one looks at the average rank of $\bar{R}^2$ and wall time for the selected datasets in Table 5 and 6 (5, 20) 5 boosting iterations with 20 bagging ensemble size seems to produce the best results.

Building on the promising results obtained with a learning rate of 1.0 and 75% feature subsampling for SGB(OZA), the following experiment summarized in Table 7 compares the two best-performing SGB(OZA) configurations, namely SGB(OZA) (10, 10) and SGB(OZA) (5, 20), against the top-performing baselines: SOKNL, OZAREG, and ARFREG, across various datasets. Both SGB(OZA) variants maintain the previously identified optimal settings: learning rate of 1.0 and 75% feature usage.

The results indicate that these SGB(OZA) configurations consistently achieve superior predictive performance, as evidenced by higher average $\bar{R}^2$ scores and better average rank. Moreover, they demonstrate notable computational efficiency, running approximately twice as fast as SOKNL, the next best method in terms of predictive accuracy.

5 Limitations

Similar to SGBT (Gunasekara et al. 2024), although SGBR employs streaming regressors with an internal Base Learner Replacement strategy, it may lack the self-tuning capability found in SOKNL.

6 Conclusions

This paper presents a novel way to leverage gradient boosting for data stream regression. This work employs bagging-based streaming regressors to overcome the high variance in streaming gradient boosted method SGBT. Bagging-based streaming regressors are employed in two ways: as base learners to existing SGBT and as an ensemble method that aggregates multiple SGBTs. The former is identified as SGB(Bag), and the latter is identified as Bag(SGBT). Even though Bag(SGBT) method OZA(SGBT) performs slightly better than SGB(Bag) method SGB(OZA), SGB(OZA) is very efficient considering the empirical evaluations. This efficient SGB(OZA) outperforms the current state-of-the-art streaming regression method SOKNL considering both predictive power and computing cost. Furthermore, empirical evaluations suggest that SGB(Bag) is able to boost the performance of traditional bagging and random subspace streaming regressors without significant computing overhead.

Appendix A experimental setup

A.1 software and hardware

1. Experiments relating to Tables 3, 4 and Figs. 4 and 5 used 5 iterations with random seeds 1, 2, 3, 4 and 5. Experiments were run using the MOA framework.
2. Experiments relating to Table 5, 6, and 7, Fig. 10 used 5 iterations with random seeds 1, 2, 3, 4 and 5. Experiments were run using the MOA framework.
3. Experiments for Fig. 6 used 3 iterations with random seeds 1,2 and 3. Experiments were run using the MOA framework.

- 4. Bias-variance computation for Figs. 2, 7 and 8 was done using the CapyMOA framework. For bias variance calculation, we used *mlxtend*'s $bias_variance_decomp$ function with MSE loss. Incremental MOA regressors were presented as sklearn regressors to $bias_variance_decomp$ using CapyMOA framework. Then, MOA regressors were incrementally trained using the given training set.Experiments 1) used multiple Ubuntu 24.04.2 LTS systems with Intel(R) Xeon(R) (3.7GHz) 20-Core Processors, with 62-125 GB RAM
- Experiments 2) used a Windows (64-bit) system with AMD Ryzen 7 3700X (3.60 GHz) 8-Core Processor, with 16.0 GB RAM
- Experiments 3) used a MacBook Pro (MacOs 14.5) system with 2.6 GHz 6-Core Intel Core i7 Processor, with 16.0 GB RAM
- Experiments 4) used an Ubuntu 18.04 LST system with AMD EPYC 7702 64-Core Processor at 4.00GHz, and with 1000GB RAM.
- Java Virtual Machine (JVM) version: JDK 1.8.0
- JVM settings: -Xms50m -Xss1g, -Xmx48g (Experiment 1 only) -Xmx12g (for all other Experiments). In Experiment 1, Sgb(Oza) ran with -Xmx12g, but other algorithms required -Xmx48g.GitHub repository https://github.com/nuwangunasekara/SGBR.git contains the code, the datasets used, and instructions on how to run the experiments.

Appendix B additional results

Acknowledgments Nuwan Gunasekara would like to acknowledge the support from the Knowledge Foundation (KKs) and Vinnova (Sweden's innovation agency).Heitor Murilo Gomes acknowledges the financial support of the Marsden Fund under award number VUW2213. Bernhard Pfahringer and Albert Bifet would like to acknowledge the support from the Time-Evolving Data Science / Artificial Intelligence for Advanced Open Environmental Science (TAIAO) project.

Author contributions N.G wrote the manuscript, developed the algorithm, and conducted experiments. B.P., H.M.G., and A.B provided guidance and reviewed the manuscript.

Funding Open access funding provided by Halmstad University.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Bifet A, Gavalda R (2007) Learning from time-changing data with adaptive windowing. In: SIAM (SDM), pp. 443–448. SIAM

Bifet A, Gavaldà R, Holmes G, Pfahringer B (2018) Machine Learning for Data Streams: with Practical Examples in MOA. MIT Press

Bühlmann P, Yu B (2000) Explaining bagging. In: Research Report/Seminar Für Statistik, Eidgenössische Technische Hochschule (ETH), vol. 92. Seminar für Statistik, Eidgenössische Technische Hochschule (ETH)

Chen S-T, Lin H-T, Lu C-J (2012) An online boosting algorithm with theoretical justifications. arXiv preprint arXiv:1206.6422

Chen T, Guestrin C (2016) Xgboost: A scalable tree boosting system. In: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining, pp. 785–794

Chicco D, Warrens MJ, Jurman G (2021) The coefficient of determination r-squared is more informative than smape, mae, mape, mse and rmse in regression analysis evaluation. Peerj computer science 7:623

Choudhary A, Jha P, Tiwari A, Bharill N (2021) A brief survey on concept drifted data stream regression. SocProS, 733–744

Domingos P (2000) A unified bias-variance decomposition for zero-one and squared loss. AAAI/IAAI 2000:564–569

Domingos P, Hulten G (2000) Mining high-speed data streams. In: Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 71–80

Fanaee-T H, Gama J (2014) Event labeling combining ensemble detectors and background knowledge. Progress in Artificial Intelligence 2:113–127

Freund Y, Schapire RE (1997) A decision-theoretic generalization of on-line learning and an application to boosting. J Comput Syst Sci 55(1):119–139

Friedman JH (2001) Greedy function approximation: a gradient boosting machine. Annals of statistics, 1189–1232

Friedman JH (1991) Multivariate adaptive regression splines. Ann Stat 19(1):1–67

Friedman JH (2002) Stochastic gradient boosting. Comput Stat Data Anal 38(4):367–378

Friedman J, Hastie T, Tibshirani R (2000) Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors). Ann Stat 28(2):337–407

Gomes HM, Barddal JP, Ferreira LEB, Bifet A (2018) Adaptive random forests for data stream regression. In: ESANN

Gomes HM, Montiel J, Mastelini SM, Pfahringer B, Bifet A (2020) On ensemble techniques for data stream regression. In: 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE

Gomes HM, Read J, Bifet A (2019) Streaming random patches for evolving data stream classification. In: IEEE (ICDM), pp. 240–249. IEEE

Gomes HM, Bifet A, Read J, Barddal JP, Enembreck F, Pfahringer B, Holmes G, Abdessalem T (2017) Adaptive random forests for evolving data stream classification. Mach Learn 106(9):1469–1495

Gomes HM, Barddal JP, Enembreck F, Bifet A (2017) A survey on ensemble learning for data stream classification. ACM (CSUR) 50(2):1–36

Gouk H, Pfahringer B, Frank E (2019) Stochastic gradient trees. In: Lee, W.S., Suzuki, T. (eds.) Proceedings of The Eleventh Asian Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 101, pp. 1094–1109. PMLR. https://proceedings.mlr.press/v101/gouk19a.html

Gunasekara N, Pfahringer B, Gomes H, Bifet A (2024) Gradient boosted trees for evolving data streams. Mach Learn. https://doi.org/10.1007/s10994-024-06517-y

Gunasekara N, Nowaczyk S, Pashami S (2025) Pragmatic paradigm for multi-stream regression. In: Krempl G, Puolamäki K, Miliou I (eds) Advances in Intelligent Data Analysis XXIII. Springer, Cham, pp 358–372

Halstead B, Koh YS, Riddle P, Pears R, Pechenizkiy M, Bifet A, Olivares G, Coulson G (2022) Analyzing and repairing concept drift adaptation in data stream classification. Machine Learning, 3489–3523

Hamidieh K (2018) A data-driven statistical model for predicting the critical temperature of a superconductor. Comput Mater Sci 154:346–354

Hulten G, Spencer L, Domingos P (2001) Mining time-changing data streams. In: ACM SIGKDD, pp. 97–106

Ikonomovska E, Gama J, Džeroski S (2011) Learning model trees from evolving data streams. Data Min Knowl Discov 23(1):128–168

Kovacs A, Bogdandy B, Toth Z (2021) Predict stock market prices with recurrent neural networks using nasdaq data stream. In: 2021 IEEE 15th International Symposium on Applied Computational Intelligence and Informatics (SACI), pp. 000449–000454. IEEE

Lobo JL, Ballesteros I, Oregi I, Del Ser J, Salcedo-Sanz S (2020) Stream learning in energy iot systems: a case study in combined cycle power plants. Energies 13(3):740

Manapragada C, Webb GI, Salehi M (2018) Extremely fast decision tree. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1953–1962

Mastelini SM, Nakano FK, Vens C, Leon Ferreira ACP et al (2022) Online extra trees regressor. IEEE Transactions on Neural Networks and Learning Systems 34(10):6755–6767

Montiel J, Mitchell R, Frank E, Pfahringer B, Abdessalem T, Bifet A (2020) Adaptive xgboost for evolving data streams. In: 2020 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE

Nash W, Sellers T, Talbot S, Cawthorn A, Ford W (1995) Abalone. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C55C7W

Oprea SV, Bâra A, Diaconita V (2019) Sliding time window electricity consumption optimization algorithm for communities in the context of big data processing. IEEE Access 7:13050–13067

Oza NC, Russell SJ (2001) Online bagging and boosting. In: International Workshop on Artificial Intelligence and Statistics, pp. 229–236. PMLR

S ARV. Demand Forecasting — kaggle.com. https://www.kaggle.com/datasets/aswathrao/demand-forecasting. [Accessed 31-10-2024]

Sun Y, Pfahringer B, Gomes HM, Bifet A (2022) Soknl: A novel way of integrating k-nearest neighbours with adaptive random forest regression for data streams. Data Min Knowl Discov 36(5):2006–2032

Sun Y, Gomes HM, Pfahringer B, Bifet A (2024) Real-time energy pricing in new zealand: an evolving stream analysis. In: Pacific Rim International Conference on Artificial Intelligence, pp. 91–97. Springer

Sun Y, Gomes HM, Pfahringer B, Bifet A (2025) Evaluation for regression analyses on evolving data streams. arXiv preprint arXiv:2502.07213

Wang K, Lu J, Liu A, Song Y, Xiong L, Zhang G (2022) Elastic gradient boosting decision tree with adaptive iterations for concept drift adaptation. Neurocomputing 491:288–304

Wright S (1921) Correlation and causation. J Agric Res 20(7):557

Wu H, Elizaveta D, Zhadan A, Petrosian O (2023) Forecasting online adaptation methods for energy domain. Eng Appl Artif Intell, 106499

Yin P, Fan X (2001) Estimating r 2 shrinkage in multiple regression: A comparison of different analytical methods. J Exp Educ 69(2):203–224

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Nuwan Gunasekara[1,2] · Bernhard Pfahringer[2] · Heitor Murilo Gomes[3] · Albert Bifet[2,4]

✉ Nuwan Gunasekara
nuwan.gunasekara@hh.se

Bernhard Pfahringer
bernhard@waikato.ac.nz

Heitor Murilo Gomes
heitor.gomes@vuw.ac.nz

Albert Bifet
abifet@waikato.ac.nz

[1] CAISR, Halmstad University, Halmstad, Sweden

[2] AI Institute, University of Waikato, Waikato, New Zealand

[3] Victoria University of Wellington, Wellington, New Zealand

[4] LTCI, Télécom Paris, IP Paris, Paris, France